

Denormalization and NoSQL Modelling

25th May 2025

R00225669

Rebecca Lordan

Denormalization

Part 1 – Denormalization

What opportunities might exist for denormalizing these relations when defining the physical records for this database?

Denormalization, copying some information from one table to another to make things faster and easier to use, even if this means repeating data. For example, instead of just having the storeID (whereWork) in the Employee table, you could also include the store's region or square footage right in the Employee table. Employee(employeeID, whereWork, employeeName, employeeAddress, storeRegion, storeSquareFeet).

You could also add the departments sales goal into the Schedule table, not just the department ID. Schedule(departmentID, employeeID, date, deparmentSalesGoal).

By doing this it can help to make the database queries faster and simpler, as there is no need to keep joining different tables each time you want the information.

Under what circumstances would you consider creating such denormalized records?

You might consider denormalizing your database if you need faster access to information like the app is mostly about displaying information where speed is important, denormalization helps to speed this up by putting all the data that you need into one place. Another circumstance you might consider denormalization is when the copied data doesn't change much, like store region which won't be updated often, its safer to have it in more than one table. If there is a lot of people reading the data at the one time by using denormalization it can help the database to handle more users and to run smoother.

Possible Foreign Keys

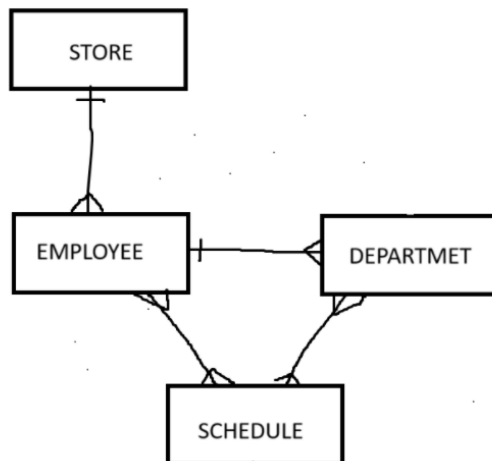
Emplyee Table – whereWork is a foreign key to Store

Deparment Table – managerID is a foreign key to Employee

Schedule Table – departmentID is a foreign key to Deparment Table

Schedule Table – employeeID is a foreign key to Employee Table

ERD Diagram



Two main places where denormalization could be applied

Employee – Store

Schedule – Employee – Department

Analyse each possibility

One possibility for denormalization would be to include store details like region and square footage directly into the Employee table.

Outcome of denormalization

Employee(employeeID(PK), whereWork, employeeName, employeeAddress, storeRegion, storeSquareFeet)

The whereWork attribute in the Employee table points to the storeID in the Store table, using store information as a reference for each employee. Before deciding whether to denormalize, you need to know stuff like how often store details like region change and whether the employees need to be linked to more than one store. If you add store details directly into the Employee table, this could lead to a lot of repeated data which takes up storage and makes it hard to keep everything consistent if a store every change its information.

Part 2 – NoSQL Modelling

1. Based on the given ERD, state an aggregate boundary

How do the users access the data, once they have it what results do we expect to get?

Three possible designs for this example

- 1) Access all data through candidate, all demographic, education, experience and training data is stored in a single JSON object per candidate.
- 2) Access all education and experience information without any knowledge of candidate personal details, personal information like name is stored separately from education and experience data.
- 3) Access all education and experience information without knowing any training or personal details, Training data is separated from both personal and education data.

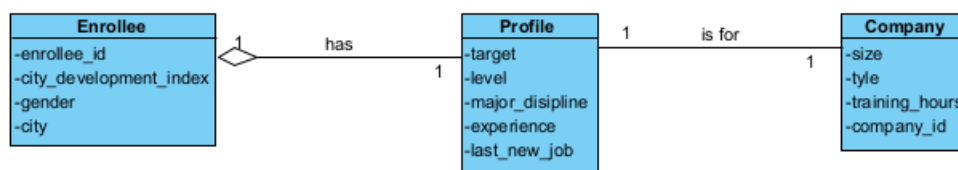
Define Aggregate Boundary

When we access the profile aggregate, we do not expect to see any candidate personal information. There are two main aggregates – candidate and profile.

The candidate aggregate contains data related to personal details like gender.

The profile aggregate consists of education, experience, company details and training hours. The profile also contains the enrolled_id as a reference to the candidate aggregate.

2. Use the visual paradigm to draw aggregate data model for the started aggregate boundary



3. Use the csv data and the aggregate data model designed in the task 2 to create JSON files. Each file only contains on JSON object.

Assumption: Assume that each profile and company are unique to a single enrollee, and that all profile and company information is always accessed together with enrollee's demographic information.

```
Enrollee.json
{
  "enrollee_id": "ID1234",
  "city": "cork",
  "city_development_index": 0.1,
  "gender": "Female",
  "target": 1,
  "level": "Master",
  "major_discipline": "Computers",
  "experience": "5",
  "last_new_job": "1",
  "company_size": "24",
  "company_type": "Public",
  "training_hours": 6,
  "company_id": 111
}
```